

Instrucciones para la realización de la prueba:

Duración de la prueba **2 h 45 m** desde el comienzo de la misma, **si combinamos un ejercicio de 7 puntos con otro de 3.**

Ejercicio 1 Curso 2023-24 90 minutos más o menos (7 puntos)

partiendo de los recursos para el examen en la carpeta `ejercicio1` donde dispones programa principal `Program.cs` para **probar tu implementación** y que te puede servir de orientación si tienes dudas en alguna de las especificaciones. Vamos a implementar un **programa que gestione el inventario de una farmacia**. Para ello, vamos a definir una abstracción de almacén genérico con las siguientes especificaciones:

1. Definir la interfaz `IProducto` cons las propiedades `Nombre` , `Precio` y `Proveedor` de solo lectura y los tipos de datos adecuados.
2. Definir la clase `DatosAlmacenaje` con las propiedades `Unidades` de tipo entero y lectura/escritura y `Ubicacion` de tipo entero y solo lectura. El constructor de la clase recibe la ubicación del producto en el almacén y establece las unidades a 0.
3. Definir la **clase parametrizada** `Almacen` donde el tipo parametrizado debe cumplir la restricción de que implemente la interfaz `IProducto` es decir **el tipo parametrizado representará algún tipo de producto**. La clase debe tener las siguientes características:
 - Una propiedad **privada** `Inventario` de tipo `Dictionary<T, DatosAlmacenaje>` y solo lectura donde `T` es el tipo parametrizado de la clase.

✦ **Nota:** Fíjate que las **claves** del diccionario son los productos y los **valores** son los datos de almacenaje del mismo, esto es, las unidades y su ubicación.

- Una propiedad **privada** `Ubicaciones` de tipo `bool[]` y solo lectura que representará si las ubicaciones fijan en el almacén están ocupadas o no. Teniendo en cuenta que:
 - **Solo podrá haber un producto por ubicación.**
 - El **índice del array representará la ubicación** que guardamos en `DatosAlmacenaje` y el valor `true` si está ocupada y `false` si está libre.

En el ejemplo siguiente, las ubicaciones 0, 1 y 2 están ocupadas y las demás libres:

```

0 1 2 3 4 5 6 7 8 9 ...
Ubicaciones -> [T][T][T][F][F][F][F][F][F][F] ...

```

- Un evento denominado `OnReponerStock` y del tipo `Action<T>?` que se generará **cuando el stock de un producto llegue a 0**. Tenlo en cuenta el el método en que retiremos productos del almacén.
- Un constructor que recibe el número de ubicaciones disponibles en el almacén e instanciara en memoria las colecciones que hemos definido como propiedades de solo lectura.
- Un método `Productos` que recibe un delegado con forma de predicado predicado y devuelve **una lista** de productos que cumplen dicho predicado que se aplicará sobre los datos de almacenaje del mismo. Esto es, me devolverá aquellos productos que cumplan un determinado stock o ubicación.

- Un método `EstaEnAlmacen` que recibe un producto y devuelve si está en el almacén o no.
- Un método `Alta` que recibe un producto y lo da de alta en el almacén. Teniendo en cuenta que:
 - Si el producto **ya está** en el almacén se lanzará una excepción de tipo `InvalidOperationException` con el mensaje **"El producto ya está dado de alta en el almacén"**.
 - Buscaremos la primera ubicación libre en el almacén y si no hay lanzaremos una excepción de tipo `InvalidOperationException` con el mensaje **"No hay ubicaciones libres en el almacén"**.
 - Si todo es correcto, añadiremos el producto al diccionario de inventario y marcaremos la ubicación como ocupada en el array de ubicaciones.
- Un método `Baja` que recibe un producto y lo da de baja del almacén. Teniendo en cuenta que:
 - Si el producto **no está** en el almacén se lanzará una excepción de tipo `InvalidOperationException` con el mensaje **"El producto no está en el almacén"**.
 - Marcaremos la ubicación como libre en el array de ubicaciones y eliminaremos el producto del diccionario de inventario.
- Un método `AgregaStock` que recibe un producto y un número de unidades y agrega stock al producto. Ten en cuenta que, no se puede agregar stock a un producto que no está en el almacén o en caso contrario se lanzará una excepción igual a la del método `Baja`.
- Un método `RetiraStock` que recibe un producto y un número de unidades y retira tantas unidades como se indique por parámetro. El método **devuelve el número de unidades retiradas**. Ten en cuenta que, no se puede retirar stock a un producto que no está en el almacén o en caso contrario se lanzará una excepción igual a la del método `Baja`. Además, no podemos retirar más unidades de las que tiene el producto en stock y por tanto el número de unidades no puede quedar negativo. Por ejemplo, si un producto tiene 10 unidades y queremos retirar 15, solo podremos retirar 10 unidades que será lo que retorne el método y se quedará el stock en 0 y por tanto disparando el evento de *'rotura'* de stock.
- Un método `ResumenInventario` que devuelve un string con un resumen del inventario en el siguiente formato:

```

Nombre          Ubicación  Unidades  Precio
-----
Paracetamol     0         10      5,99
Ibuprofeno      1          0      7,49
Amoxicilina     2         10      9,99
-----
Valor total: 159,80

```

🚩 **Nota:** Nombre ocupa 20 caracteres, Ubicación 10, Unidades 10 y Precio 10.

- Un método `ExportarACSV` serializa el inventario a un **fichero CSV en UTF-8** con el siguiente formato:

```

Nombre,Proveedor,Precio,Ubicación,Unidades
Paracetamol,Proveedor A,"5,99 €",0,0
Amoxicilina,Proveedor C,"9,99 €",2,10

```

- Define la clase `Medicamento` que implementa la interfaz `IProducto` y es un **record** (registro).

🚩 **Nota:** Si no sabes definir un record y lo defines como clase normal, recuerda que al usarse como clave en un diccionario en ese caso debes sobrescribir los métodos `Equals` y `GetHashCode`.

- Define la clase `Farmacia` con las siguientes características:

- Una propiedad **privada** `Nombre` de tipo `string` y solo lectura.
- Una propiedad **privada** `Almacen` de tipo almacén de medicamentos.

- Un constructor que recibe el nombre de la farmacia y lo almacena en la propiedad correspondiente y crea un almacén de medicamentos con 100 ubicaciones disponibles. Además, el constructor se suscribe al evento `OnReponerStock` del almacén a través de un **método privado** denominado `AvisoReposicionMedicamento` que mostrará un mensaje por consola como a este:

AVISO: Mandando reposición stock de Paracetamol al proveedor Proveedor A

- Los métodos `Alta`, `Baja`, `Reponer` y `Retirar` como **'cuerpo de expresión'** que delegan en el almacén de medicamentos.
- Un método `MedicamentosAgotados` que devuelve una lista de medicamentos que tienen 0 unidades en stock.
- Un método `MostrarInventario` que devuelve un string con el nombre de la farmacia y un resumen del inventario del almacén.
- Un método `ExportarInventarioCSV` que recibe una ruta de fichero y delega en el almacén la serialización del inventario a CSV.

Criterios de evaluación:

✦ **Nota:** Cualquier error de sintaxis en la definición de las clases o métodos que genere errores de compilación, así como el código a medio implementar podrá suponer la calificación con **0 pts** puntos del apartado correspondiente.

- (✓ 0,25 pts) Definir interfaz `IProducto`.
- (✓ 0,25 pts) Definir la clase `DatosAlmacenaje`.
- (✓ 0,5 pts) Definir la clase `Almacen` con sus propiedades eventos y constructor.
- (✓ 0,75 pts) Implementar el método `Productos`.
- (✓ 0,75 pts) Implementar el método `Alta`.
- (✓ 0,5 pts) Implementar el método `Baja`.
- (✓ 0,5 pts) Implementar el método `AgregaStock`.
- (✓ 0,5 pts) Implementar el método `RetiraStock`.
- (✓ 0,5 pts) Implementar el método `ResumenInventario` que además de mostrar los datos indicados debe hacerlo en el formato especificado.
- (✓ 0,5 pts) Implementar el método `ExportarACSV` que debe generar un fichero con el formato especificado.
- (✓ 0,25 pts) Definir el record `Medicamento`.
- (✓ 0,25 pts) Definir la clase `Farmacia` con sus propiedades, constructor y método de suscripción al evento del almacén.
- (✓ 0,25 pts) Implementar los métodos `Alta`, `Baja`, `Reponer` y `Retirar`.
- (✓ 0,5 pts) Implementar el método `MedicamentosAgotados`.
- (✓ 0,25 pts) Implementar el método `MostrarInventario`.
- (✓ 0,25 pts) Implementar el método `ExportarInventarioCSV`.
- (✓ 0,25 pts) Ejecución correcta del programa de prueba en `Program.cs`.

Ejercicio 2 curso 2024-25 80 minutos más o menos (7 puntos)

A partir de los recursos proporcionados en el 'workspace' `ejercicio2`, donde podrías encontrar clases ya implementadas y un programa principal `Program.cs` para **probar tu implementación**, vamos a desarrollar un **sistema simplificado para gestionar el catálogo y los préstamos de una biblioteca**. Para ello, definiremos un catálogo genérico parametrizado con las siguientes especificaciones:

1. Definir la interfaz `IItemCatalogo<T>` con la propiedad de solo lectura `Id` del tipo parametrizado `T`.
Donde `T` debe cumplir la restricción de ser `IEquatable<T>` e `Id` representará un **identificador único** para cada elemento del catálogo y el tipo puede variar si el catálogo es de **libros, sellos, vinilos**, etc.
2. Se te proporciona **ya implementado** el **record** `DatosPrestamo` que guarda `IdSocio`, `FechaPrestamo` y `Dias` donde esta última propiedad representa los días máximos desde la fecha del préstamo para que expire el mismo. Además, fíjate que hay una propiedad calculada `Vencido` que me indicará si el préstamo ha expirado o no.
3. Dispones también de la clase **ya implementada** `CatalogoException` para generar excepciones personalizadas en la próxima clase a implementar.
4. Definir la **clase parametrizada** `CatalogoPrestable<T>` donde el tipo parametrizado `T` debe cumplir la restricción de implementar la interfaz `IEquatable<T>`. En este caso `T` un tipo de ítem gestionado en el catálogo.
 - Una **propiedad privada** `Items` de tipo `List<T>` y solo lectura, que contendrá todos los elementos que contiene el catálogo.
 - Una **propiedad pública** `Inventario` que me devolverá los ítems como `IEnumerable<T>`, de esta manera desde fuera solo podré recorrerlos, pero nunca añadir y borrar en la lista privada que solo se hará desde dentro de la clase.
 - Una **propiedad privada** `PrestamosActivos` de tipo `Dictionary<T, DatosPrestamo>` y solo lectura. Las **claves** serán los elementos del catálogo prestados y los **valores** serán los datos del préstamo, del tipo `DatosPrestamo` que se te proporciona.
 - Un **método privado** `EnCatalogo` que recibe un ítem del catálogo y retornará un booleano que indique si se encuentra o no en el catálogo.
 - Un **método público** `Prestado` que recibe un ítem del catálogo y retornará un booleano que indique si se encuentra o no prestado, esto es, entre las claves de `PrestamosActivos`.
 - Un **constructor por defecto** que inicialice las colecciones (`Items` y `PrestamosActivos`).
 - Un **método público** `Agrega` que recibe un ítem del catálogo y lo añade a `Items`.
 - Si el elemento está en el catálogo, se lanzará una excepción `CatalogoException` con el mensaje "El elemento ya existe en el catálogo."
 - Un **método público** `Retira` que recibe un ítem del catálogo y lo retira de `Items`.
 - Si el elemento no está en el catálogo, se lanzará una excepción `CatalogoException` con el mensaje "El elemento no se encuentra en el catálogo."
 - Si el elemento está prestado, se lanzará una excepción `CatalogoException` con el mensaje "No se puede eliminar un elemento que está prestado."
 - Un **método público** `Presta` que recibe un ítem del catálogo, un `IdSocio` (string), una `Fecha` (DateOnly) y un número de días de préstamo (opcional, por defecto 30).
 - Si el elemento no está en el catálogo, se lanzará una excepción `CatalogoException` con el mensaje "El elemento no se encuentra en el catálogo."
 - Si el elemento ya está prestado, se lanzará una excepción `CatalogoException` con el mensaje "El elemento ya está prestado."
 - Si todo es correcto, se añade al diccionario de préstamos activos.
 - Un **método público** `Devuelve` que recibe un ítem del catálogo y lo elimina de `PrestamosActivos`.

- Si el elemento no está prestado, se lanzará una excepción `CatalogoException` con el mensaje "El elemento no estaba prestado."
 - Un **método público** `DatosPrestamo` que recibe un ítem del catálogo y devuelve los datos del préstamo (de tipo `DatosPrestamo`) si el elemento está prestado, o `null` si no lo está.
 - Una **propiedad pública** `PrestamosVencidos` que devuelve una secuencia de tuplas (`IEnumerable<(T, DatosPrestamo)>`) obtenidas a partir de `PrestamosActivos` donde cada tupla contenga el elemento prestado y sus datos de préstamo, pero solo aquellos donde la propiedad `Vencido` de `DatosPrestamo` se evalúe a `true`. La secuencia debe estar ordenada por la fecha de préstamo.
5. Define la clase `Libro` que implemente las interfaces `IItemCatalogo<Isbn>`, `IEquatable<Libro>` donde `Isbn` es un tipo `record` que ya te te proporciona implementado. La implementación del `Equals` comparará por el `Id` que recuerda que implementa `Equals` por restricción.
6. Define la clase `Biblioteca` con las siguientes características:
- Un **campo privado constante** de tipo entero denominado `MAXIMO_DIAS_PRESTAMO` que tendrá valor 60.
 - Una **propiedad privada** `Nombre` de tipo `string` y solo lectura.
 - Una **propiedad privada** `CatalogoLibros` de tipo `CatalogoPrestable<Libro>`.
 - Un constructor que recibe el nombre de la biblioteca, lo almacena y crea la instancia `CatalogoLibros`.
 - Métodos públicos `RegistrarLibro`, `RealizarPrestamo`, `RecibirDevolucion` que reciban los parámetros necesarios y deleguen la lógica en los métodos correspondientes de `CatalogoLibros`. Usa la sintaxis de *'cuerpo de expresión'* si lo consideras adecuado.
 - Un **método público** `BuscarLibrosPorTitulo` que usando la propiedad `Inventario` del catálogo devuelva un `IEnumerable<Libro>`.
 -  **Tip:** Puedes usar el método de `string` `Contains` junto con `StringComparison.OrdinalIgnoreCase` o expresiones regulares para la búsqueda.
 - Un **método público** `IEnumerable<Libro> LibrosPrestadosA(string idSocio)` que devuelve un `IEnumerable<Libro>`.
 -  **Tip:** Puedes usar la propiedad `Inventario` y el método `DatosPrestamo` del catálogo para realizar la operación
 - Un **método público** `InformeInventario` que devuelva un `string` con los siguientes datos y formato:
 - **Las columnas miden 30, 30 y 12 respectivamente**

```

Inventario Biblioteca Central ...
Título                               Autor                               Estado
-----
Cien años de soledad                 Gabriel García Márquez             Disponible
El nombre del viento                 Patrick Rothfuss                   Disponible
El temor de un hombre sabio          Patrick Rothfuss                   Disponible
Fundación                            Isaac Asimov                       Disponible
-----
Total Elementos: 4

```

- Un **método público** `InformePrestamosVencidos` que genere un archivo de **markdown en UTF-8** con la siguiente tabla:
 - Las columnas miden 30, 14, 14, 4 respectivamente.

****Fecha de generación del informe**:** 01/05/2025

Titulo	ID Socio	Fecha Prestamo	Dias
-----	-----	-----	----
El nombre del viento	S001	17/01/2025	117
Cien años de soledad	S001	04/02/2025	99

Usa la propiedad `PrestamosVencidos` del catálogo para obtener los libros que llevan más de 60 días prestados y generar el informe con el formato indicado. Para calcular los días de préstamo puedes usar la propiedad `FechaPrestamo` del objeto `DatosPrestamo` y la fecha actual. Además debes utilizar un `StreamWriter` para escribir el archivo de markdown.

El archivo se guardará con el nombre `InformePrestamosVencidos.md` y si ya existe se añadirán los nuevos registros al final del mismo sin eliminar los anteriores, pero actualizando la fecha de generación del informe y dejando una línea en blanco entre el último registro y la fecha de generación del informe.

7. Por último, se te proporciona un código **comentado** en el fuente `Program.cs` que deberás ir descomentando conforme implementes los métodos que se usan para probar tu implementación.

Criterios de evaluación:

🚩 **Nota:** Cualquier error de sintaxis en la definición de las clases o métodos que genere errores de compilación, así como el código a medio implementar podrá suponer la calificación con **0 pts** puntos del apartado correspondiente.

- (✓ **0,5 pts**) Interfaz `IItemCatalogo<T>` .
- (✓ **0,25 pts**) Propiedades `Items` , `Inventario` y `PrestamosActivos` en `CatalogoPrestable<T>` .
- (✓ **0,25 pts**) Métodos `EnCatalogo` , `Prestado` y constructor en `CatalogoPrestable<T>` .
- (✓ **0,5 pts**) Métodos `Agrega` , `Retira` en `CatalogoPrestable<T>` .
- (✓ **0,5 pts**) Métodos `Presta` , `Devuelve` en `CatalogoPrestable<T>` .
- (✓ **0,5 pts**) Método `DatosPrestamo` en `CatalogoPrestable<T>` .
- (✓ **0,25 pts**) Propiedad `PrestamosVencidos` en `CatalogoPrestable<T>` .
- (✓ **0,5 pts**) Implementación clase `Libro` .
- (✓ **0,5 pts**) Implementación campos, propiedades y constructor propuestos en `Biblioteca` .
- (✓ **0,5 pts**) Implementación de `RegistrarLibro` , `RealizarPrestamo` , `RecibirDevolucion` en `Biblioteca` .
- (✓ **0,5 pts**) Implementación del método `BuscarLibrosPorTitulo` en `Biblioteca` .
- (✓ **0,5 pts**) Implementación del método `LibrosPrestadosA` en `Biblioteca` .
- (✓ **0,5 pts**) Implementación del método `InformeInventario` en `Biblioteca` .
- (✓ **0,75 pts**) Implementación del método `InformePrestamosVencidos` en `Biblioteca` .
- (✓ **0,5 pts**) En función del código descomentado y ejecutado en `Program.cs` .

Ejercicio 3 curso 2023-24 35 minutos más o menos (3 puntos)

A partir de los recursos para el examen en la carpeta `ejercicio2` completa el código del programa principal

`Program.cs` en las zonas comentadas para así completar las cinco consultas que en él se piden y que describimos a continuación:

( **0,5 puntos**) por consulta si la salida por consola se corresponde con la propuesta.

- **Consulta 1:** Mostrar título, año, país y valoración de las películas españolas del año 2014 ordenados descendientemente por valoración.

Salida por consola:

```
{ Titulo = Ocho apellidos vascos, Año = 2014, Pais = España, Valoracion = 6,95 }
{ Titulo = El Niño, Año = 2014, Pais = España, Valoracion = 6 }
```

- **Consulta 2:** Mostrar título, año, país y duración de las películas con duración entre 100 y 120 minutos, es decir, con duración mayor o igual de 100 y menor o igual de 120.

Salida por consola:

```
{ Titulo = El pasajero, Año = 2018, Pais = Estados Unidos, Duracion = 105 }
{ Titulo = Venganza bajo cero, Año = 2019, Pais = Reino Unido, Duracion = 118 }
{ Titulo = El rey león, Año = 2019, Pais = Estados Unidos, Duracion = 118 }
{ Titulo = Toy Story 4, Año = 2019, Pais = Estados Unidos, Duracion = 100 }
```

- **Consulta 3:** Mostrar título, año y géneros" de las películas que el genero sea de "*drama*", "*romance*" o "*fantástico*".

 **Nota:** Debes hacerlo completando el cuerpo de expresión de método

```
static IEnumerable<Consulta3Dto> Consulta3() => ... proporcionado en el fuente.
```

Salida por consola:

```
Consulta3Dto { Titulo = El Niño, Año = 2014, Generos = [intriga, accion, drama, policiaca] }
Consulta3Dto { Titulo = Ocho apellidos vascos, Año = 2014, Generos = [comedia, romance, crimen] }
Consulta3Dto { Titulo = Aquaman, Año = 2018, Generos = [accion, fantastico] }
Consulta3Dto { Titulo = El rey león, Año = 2019, Generos = [animacion, aventuras, drama] }
```

- **Consulta 4:** Mostrar título, país y reparto de las películas en las que conste un reparto con 2 actores.

Salida por consola:

```
{ Titulo = Todos los caminos, Pais = España, Reparto = [Dani Rovira, Clara Lago] }
{ Titulo = El pasajero, Pais = Estados Unidos, Reparto = [Liam Neeson, Patrick Wilson] }
```

- **Consulta 5:** Calcular la duración media de las películas de cada país.

 **Nota:** Se debe mostrar "*Pais: DuracionMedia*"

Salida por consola:

```
España: 107,5
Estados Unidos: 111,2
Reino Unido: 118
```

- **Consulta 6:** Muestra sin repeticiones todos géneros de las películas en cartelera usando SelectMany.

Salida por consola:

```
[intriga, crimen, accion, drama, policiaca, comedia, romance, documental, fantastico, animacion, aventuras]
```

 Importante:

- En `Cartelera.cs` de la propiedad estática `Cartelera.Peliculas` con la secuencia con la que realizar las consultas. **Analiza detenidamente su contenido y estructura** para saber realizar las mismas.
- Cada consulta debe implementarse en una única expresión.
- Puedes usar `Select` , `SelectMany` , `Where` , `OrderBy` , `OrderByDescending` , `GroupBy` , `Count` y `Distinct` .
- Puedes definir usar clases anónimas para realizar las mapeos o proyecciones salvo que se especifique otra cosa.

Ejercicio 4 curso 2024-25 35 minutos más o menos (3 puntos)

A partir de los recursos proporcionados en el 'workspace' `ej2` . donde se definen las clases `Repuesto` , `Fabricante` y el set de datos en `Inventario.cs` , completa el código del programa principal `Program.cs` en los métodos estáticos correspondientes a las **6 consultas** que se piden y que describimos a continuación:

( **0,5 puntos**) por consulta **solo si la salida por consola se corresponde con la propuesta.**

 **Importante:** Fíjate bien en el set de datos y la distinción de **fabricante**, **marca** y **modelo** de los requerimientos de las consultas.

- **Consulta 1:** Mostrar Nombre,Codigo, MarcaVehiculo y Precio de los repuestos del grupo Volkswagen con stock mayor a 100, ordenados descendientemente por precio.

Salida por consola esperada:

```
{ Nombre = Filtro de Aceite, Codigo = FA001VW, MarcaVehiculo = Volkswagen, Precio = 12,5 }
{ Nombre = Bujía, Codigo = BJ001VW, MarcaVehiculo = Volkswagen, Precio = 8,99 }
```

- **Consulta 2:** Mostrar Nombre, Codigo y Stock de los repuestos compatibles con el modelo 'Golf'.

Salida por consola esperada:

```
{ Nombre = Filtro de Aceite, Codigo = FA001VW, Stock = 150 }
{ Nombre = Pastillas de Freno Delanteras, Codigo = PFD001VW, Stock = 80 }
{ Nombre = Bujía, Codigo = BJ001VW, Stock = 200 }
{ Nombre = Amortiguador Delantero, Codigo = AD001VW, Stock = 50 }
{ Nombre = Correa de Distribución, Codigo = CD001VW, Stock = 30 }
```

- **Consulta 3:** Muestra fabricante y sus marcas sin repeticiones.

 **Nota:** Debes hacerlo completando el cuerpo de expresión de método
`static IEnumerable<ConsultaRepuestoDto> DatosConsulta3() => ...` proporcionado en el fuente.

Salida por consola esperada:

```
Volkswagen: [Volkswagen, Audi, SEAT, Skoda]
Stellantis: [Peugeot, Citroen, Opel, Fiat]
```


- **Consulta 4:** Agrupando por nombre del repuesto, muestra todos los modelos que tenemos en el inventario tengan stock o no.

Mostrar -> "Nombre: [Modelo1, Modelo2, ...]"

Salida por consola esperada:

```
Filtro de Aceite: [Golf, Polo, Tiguan]
Pastillas de Freno Delanteras: [Golf, Passat, 308, 508]
Bujía: [Golf, Polo, Jetta, C3, C4, Berlingo]
Filtro de Aire: [208, 308, 2008, A3, Q3, Tiguan]
Amortiguador Delantero: [A3, A4, Golf, Corsa, Astra, 208]
Sensor de Temperatura: [Ibiza, Leon, Polo, 500, Panda, C3]
Correa de Distribución: [Octavia, Fabia, Golf]
```

- **Consulta 5:** Nombre, Código, Marca y Stock de los repuestos de Stellantis con stock menor a 5 ordenados por Stock de menos a mayor.

Salida por consola esperada:

```
{ Nombre = Pastillas de Freno Delanteras, Código = PFD001ST, MarcaVehículo = Peugeot, Stock = 0 }
{ Nombre = Sensor de Temperatura, Código = ST001ST, MarcaVehículo = Fiat, Stock = 4 }
```

- **Consulta 6:** Muestra ordenados y sin repeticiones todos los modelos de vehículos compatibles para repuestos del grupo Volkswagen usando SelectMany.

Salida por consola esperada:

```
A3, A4, Fabia, Golf, Ibiza, Jetta, Leon, Octavia, Passat, Polo, Q3, Tiguan
```

👉 Importante:

- En **Inventario.cs** encontrarás la propiedad estática **Inventario.Repuestos** con la secuencia con la que realizar las consultas. **Analiza detenidamente su contenido y estructura** para saber realizar las mismas.
- Cada consulta debe implementarse en una única expresión LINQ donde sea posible.
- Puedes usar los operadores LINQ que consideres necesarios de entre los vistos en clase (ej: **Select**, **SelectMany**, **Where**, **OrderBy**, **OrderByDescending**, **GroupBy**, **Average**, **Contains** y **Distinct**).
- Puedes definir y usar clases anónimas para realizar las proyecciones o mapeos, salvo que se especifique el uso de un DTO (Data Transfer Object) como en la **Consulta3**.

Ejercicio 5 curso 2023-2024 100 minutos más o menos (7 puntos)

Vamos a implementar una gestión de 'carro de compra' y pago simplificada aplicando el "patrón Strategy".

👉 **Importante:** Para ello implementaremos los tipos que se describirán a continuación teniendo en cuenta las siguientes directrices:

1. Las propiedades serán **auto-implementadas** y **públicas** de **solo lectura** si no se especifica nada más.
2. Se definirán **cuerpos de expresión** siempre que sea posible.
3. Las propiedades deben inicializarse donde correspondan y nunca dejarlas a su valor por defecto.

De no cumplirse penalizará en el apartado correspondiente.

Clase **Producto** con las propiedades **codigo** de tipo entero, **descripcion** de tipo cadena y **precio** de tipo double y el constructor correspondiente.

- Debe implementar el interfaz **IEquatable<T>** para comparar productos por código.
- Invalidación también el método **ToString** para que muestre el código, la descripción y el precio.

✦ **Nota:** Fíjate como se muestran las líneas de producto en el ejemplo de ticket al final de este ejercicio, para dar un formato similar.

Clase **CarritoCompra** con la propiedad **privada productos** que será una lista de productos.

- Invalida el método **ToString** para que muestre los productos de la lista separados por un salto de línea usando a su vez el método **ToString** de cada producto.
- Método con **interfaz fluido** **Añade** al que le llegue un producto y lo añada a la lista.
- Propiedad calculada **Total** que me devuelva el total a pagar por los productos en el carrito.
- Método **Borra** al que le llegue **código de producto** y lo borre de la lista. Devolviendo un booleano que indique si lo ha podido borrar con éxito o no.

✦ **Nota:** El programa principal que se te da para probar el programa te puede servir de guía para saber si has implementado correctamente esta clase.

Interfaz **IMedioDePago** que implementará:

- Método **EstableceDatosDePago** que decidirá en su concreción el medio de pago de un determinado tipo. Por ejemplo, si es un pago con tarjeta, qué tarjeta en concreto es. Ni recibe, ni retorna nada.
- Método **ProcesaPago** que recibe la cantidad a pagar y hará el pago con el medio establecido (si lo hubiese).
- Propiedad **HayDatosDePago** booleana de solo lectura que me dice si hay algún medio de pago establecido o no.
- Propiedad **DatosDePago** string de solo lectura que me devuelve una cadena con la información del medio de pago establecido.

Clase **PagoException** que concrete las excepciones de pago.

Clase **TarjetaDeCredito** que contendrá los datos de una **tarjeta de crédito**. Por tanto contendrá las siguientes propiedades: **Titular**, **Numero**, **CVV** de tipo cadena, **Caducidad** de tipo **DateTime** y **Límite** de tipo double. Además, definiremos las propiedades de tipo double **CreditoConcedido** que será privada para modificación y la propiedad **calculada CreditoDisponible** que se calculará restando al límite el crédito concedido.

- Invalida **ToString** para que se muestre solo el **número** de la tarjeta.
- Define un **constructor privado** que inicialice los objetos tarjeta de acuerdo a los datos que hay en el fichero **tarjetas.csv**.

✦ **Nota:** El fichero **tarjetas.csv** puedes encontrarlo en el *'workspace'* del proyecto del ejercicio 2 que se te proporcionó en los recursos.

- Método de clase **LeeCSV** que **devuelva una instancia** de tarjeta de crédito a partir de la línea actual donde nos encontremos en un **StreamReader** a el fichero **tarjetas.csv** y que recibirá como parámetro tal. El método supone que estamos situados en la línea correcta del fichero y podemos leerla hasta el final.
- Método **Cargo** que se encargará de gestionar un cargo en la tarjeta de crédito. Por lo que recibirá una cantidad de dinero que cargará en el crédito concedido si se cumplan las siguiente precondiciones:
 - La tarjeta no esté caducada.
 - Haya crédito disponible para el cargo en la tarjeta.

En caso de no cumplirlas, generará una excepción de tipo `TarjetaDeCreditoException` con el mensaje correspondiente.

Si el cargo se ha realizado correctamente se mostrará una mensaje parecido a este:

`'Cargando 143,00 euros en tarjeta de crédito número 4868470817885831'`

Clase `CuentaPayPal` análoga a la anterior y que **se te proporciona junto a los recursos de este examen**. Contendrá datos de una **cuenta de PayPal**. En ella, puedes encontrar el método de clase `Lee` que realiza la lectura en formato texto con codificación UTF-8 de los datos de las cuentas de PayPal que se guardarán en el fichero `paypal.txt` en el programa principal que se te proporciona.

Clase `PagoConTarjetaDeCredito` que implementará el interfaz `IMedioDePago` descrito anteriormente.

- Definirá la propiedad privada de acceso y modificación `Tarjeta` de tipo `TarjetaDeCredito`.
- El método `EstableceDatosDePago` en esta clase:
 - i. Mostrará el mensaje `'Seleccionando tarjeta de crédito ...'`
 - ii. Leerá todas las tarjetas en `tarjetas.csv` en una lista y establecerá la propiedad `Tarjeta` a una de las tarjetas al azar de dicha lista.
 - iii. Controlará cualquier tipo de `IOException` que se produzca mostrando el mensaje correspondiente.
- El método `ProcesaPago` generará una excepción `PagoException` con el mensaje `'Error tarjeta de crédito no seleccionada'`, si no hay medio de pago y si lo hay, cargará en la tarjeta de crédito el pago controlando cualquier excepción generada por la tarjeta de crédito.
- El método `DatosDePago` generará una excepción `PagoException` con el mensaje `'Error tarjeta de crédito no seleccionada'`, si no hay medio de pago y si lo hay mostrará la tarjeta de crédito como cadena.

✦ **Nota:** Este método se considerará correcto solo si se 'refactorizas' el código para que no se repita el texto del error.

Clase `PagoConPayPal` que implementará también el interfaz `IMedioDePago`.

- Definirá la propiedad privada de acceso y modificación `CuentaPayPal` de tipo `CuentaPayPal`.
- El método `EstableceDatosDePago` en esta clase:
 - i. Mostrará el mensaje `'Seleccionando cuenta de PayPal ...'`
 - ii. Leerá todas las cuentas almacenadas `paypal.bin` en una lista y establecerá la propiedad `CuentaPayPal` a una de las cuentas al azar de dicha lista.

✦ **Nota:** El fichero `paypal.bin` cada vez que se ejecuta el programa. Puedes verlo en el programa principal que se te proporciona como recurso.

- iii. Controlará cualquier tipo de `IOException` que se produzca mostrando el mensaje correspondiente.
- El resto de la implementación es análoga a la de `PagoConTarjetaDeCredito` salvo que los cargos en las cuentas de PayPal no generan ningún tipo de excepción.

Por último, vamos a definir la clase `Pedido` que:

- Contendrá las siguientes propiedades públicas de acceso y privadas de modificación si no se especifica otra cosa:
 - i. `Id` de solo lectura y tipo entero.
 - ii. `Carrito` de solo lectura y tipo `CarritoCompra`.
 - iii. `Dirección` de tipo cadena.
 - iv. `MedioPago` **privada** de solo lectura y cuyo tipo será la estrategia o generalización que definimos para hacer pagos.

- v. **Pagado** y **Enviado** que serán de tipo booleano y que al crearse un pedido estarán a falso, siendo ambas privadas para modificación.
- Constructor que inicialice un objeto pedido a su estado inicial de acuerdo al programa principal que se te proporciona para probar las clases.
- Método **Paga** que realizará el siguiente proceso.
 - i. Si el pedido ya está pagado, generará una excepción de tipo **PedidoException** indicando el id del pedido que ha producido el error.
 - ii. Si no hay datos de pago en el medio de pago, los establecerá.
 - iii. Procesará el pago del pedido a partir del total del carrito.
 - iv. Marcará el pedido como pagado.
 - v. Mostrará la siguiente información...

```
Pegando pedido {Id}:
{Carrito}
{Mensaje que genera el establecer el medio de pago}
{mensaje que genera el pago}
El pago del pedido {Id} se ha realizado con éxito.
Procesando envío a {Direccion}...
```

Si pruebas el **programa principal que se te proporciona en recursos**. El primer pedido debería generar algo como ...

 **Importante:** Debes pensar bien los WriteLine dentro de **Paga** para que se muestren los mensajes correctamente. Teniendo en cuenta que al procesar pagos se generarán mensajes por pantalla-

```
Pegando pedido 1:
0001 Disco Duro WD                50,00E
0002 Fuente Alimentación Crossair  75,00E
0003 Ratón HP                      9,00E
0003 Ratón HP                      9,00E
Total: 143 euros
Seleccionando tarjeta de crédito ...
Cargando 143,00 euros en tarjeta de crédito número 4868470817885831
El pago del pedido 1 se ha realizado con éxito.
Procesando envío a C. Cerámica nº 24, 03010, Alicante...
```

Criterios de evaluación:

🚩 **Nota:** Cualquier error de sintaxis en la definición de las clases o métodos que genere errores de compilación, así como el código a medio implementar podrá suponer la calificación con **0 pts** puntos del apartado correspondiente.

- (✓ **0,5 pts**) Implementación de clase **Producto** .
- (✓ **1 pts**) Implementación de clase **CarritoCompra** .
- (✓ **1 pts**) Implementación de la interfaz **IMedioDePago** .
- (✓ **1 pts**) Implementa la clase **TarjetaDeCredito** .
- (✓ **1 pts**) Implementa la clase **PagoConTarjetaDeCredito** .
- (✓ **1 pts**) Implementa la clase **PagoConPayPal** .
- (✓ **1 pts**) Implementa la clase **Pedido** .
- (✓ **0,5 pts**) Ejecución sin errores de sintaxis y compilación del programa principal.